Title and Date

Document Name: Project 2: Cyclical Executive Scheduler
Date of Publication: September 8th, 2026

Lead Engineer: Niccolo James Capones

Stakeholders:
Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA
MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

High Level Description: This document is the project document for Project#2 of the CMPE311 class. It contains customer, technical and testing requirements as well as the design and results of the validation testing.

Description: The customer previously requisitioned an asynchronous system built on top of the Arduino Uno R3 platform. The product should allow a user to specify an LED number and its blinking interval. The blinking should continue asynchronously without any added input from the user. High-level & technical requirements have been derived from the customer demands and constraints. An ECO was submitted by Professor Kidd who requested to replace the Asynchronous tasking with a Cyclical Executive Scheduler. Documentation within the report will include testing, validation, sample code and videos of testing.

Result Summary: After the engineer developed a successful asynchronous system, a system involving cyclical tasking was successfully incorporated.

Requirements

Customer Requirements:

C1. The User must be able to set the blinking rate of two different LEDS
C2. System must be able to run on the Arduino Uno R3 platform
C2. Blink rate of an LED must be expressed in milliseconds

High Level Requirements:

HL1. The user through the IDE serial monitor should be able to set the blink rate of the LEDs *(C1)*

HL2. The two LEDs must run independently of each other *(C2)*

HL3. The blinking of the LEDs must not be interrupted by the User setting another rate *(C2)*

HL4. The User will use the IDE's serial monitor to determine the blink rate in N milliseconds *(C3)*

HL5. The system must use a cyclical executive task scheduler *(ECN 1.0)*

Design

Design Prerequisites:

1. Arduino Uno R3 clone
2. Arduino IDE
3. Wires
4. Power supply (5V or Laptop)

Design Requirements:

Global:

D1.1: Make Ports a ~~constant integer variable~~ #define'd macro
D1.2: Define helper function ~~blink()~~ task_blink()
D1.3: Save LED state for each LEDs
D1.4: Define and save boolean awaitingInterval
D1.5: Define and save blink intervals for both LEDs
D1.6: Create TaskType struct
D1.7: Create Task block made of TaskType structs
D1.8: Create a function to return the amount of tasks

D1.9: Create a helper function task_serial()
D1.10: Create an input buffer
D1.11: make BUF_SIZE a #define'd macro
D1.12: Global int var bufInd

In the loop:

~~D2.1: Call blink()~~
~~D2.2: Serial read for char '1' or '2' if !awaitingInterval; determines which LED~~
~~D2.3: Serial read for interval a value in ms and assign interval to chosen LED~~
~~D2.4: Scrub the monitor and prepare for the loop again~~

D2*.1: Get number of tasks
D2*.2: Get current runtime in ms
D2*.3: Use a for loop to iterate through the tasks at each interval
D2*.4: Update the last run time of the previous task
D2*.5: Call the next task

~~blink()~~ task_blink()
D3.1 For each LED…

- D3.1.1: Rate must be greater than zero
- D3.1.2: compare rate to runtime in ms; variable called currentMillis
- D3.1.3: save current blink time to the previous blink time
- D3.1.4: digitalWrite to LED pin
- D3.1.5: alternatively if rate is 0, the LED turns off or stays off

task_serial()
D4.1 While Serial.available() > 0

- D.4.1.1: use char ch to read values into the buffer
- D4.1.2: Serial read for int 1 or 2 if !awaiting interval to determine LED
- D4.1.3: Serial read for an integer value in ms and assign to selected LED
- D4.1.4 Reset the buffer after each input

Struct TaskType:
D5.1: Save task interval
D5.2: Save task run time
D5.3: define in house void function pointer

Tasks[]:
D6.1: Hold all Task structs in an array

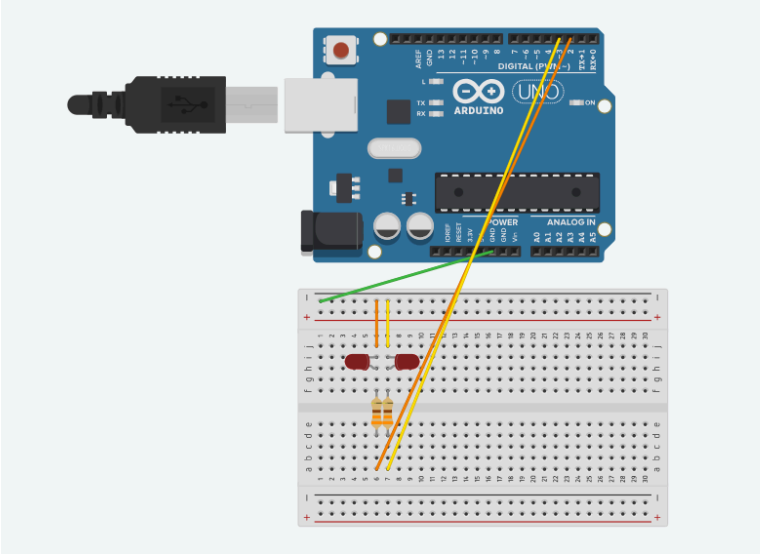


Fig. 8) Embedded System Diagram

Testing and Validation

Testing Pre-requisites

- See Design Requirements
- A sample user

In terms of testing, the engineer will have to establish the following test cases: Normal Case: Edge Case and Fail Case. The normal case (T1) would be the application of intervals such as 1000ms and 2000ms and to observe the results. If this is successful a new interval can be selected to demonstrate the asynchronous nature of the system; the blinking should not be interrupted by the serial communication. In the observation of this test, we must see the LEDs blink independently from the serial communication.

The edge case (T2) can be turning the LEDs off by passing a zero to either of them. Lastly, the demonstration of failure handling (T3) should come from choosing an invalid LED number and being returned an error message and resetting the loop.

Testing Documentation

Test	Result
T1	Attempt One: Input would not read; type matching error Attempt Two: LEDs refused to flash, added atoi() conversion Attempt Three: Successful

T2	Successful, the blinking stopped and the serial prompt reappeared
T3	Successful, the system looped back to the start despite the misinput

Summary of Attached Documents

In reference to the Github Repo, there exists a .INO file containing my code, and a video regarding documentation of the test cases.